Solution: Max of All Subarrays of Size 'k'

Problem Statement

Given an integer array and an integer `k`, design an algorithm to find the maximum for each and every contiguous subarray of size `k`.

Examples:

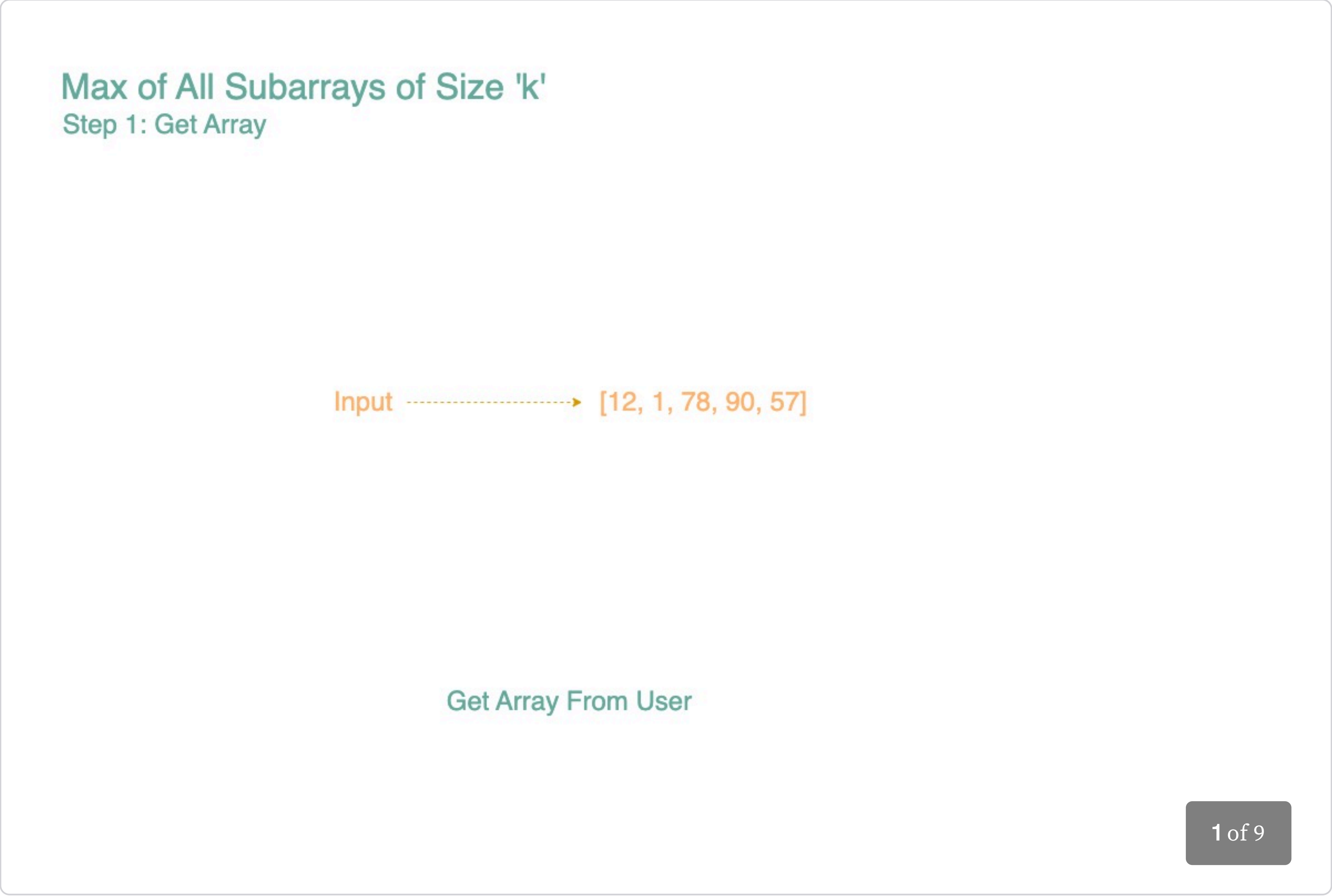
1. **Input:** array = `[1, 2, 3, 1, 4, 5, 2, 3, 6]`, k = 3
   Output: `[3, 3, 4, 5, 5, 5, 6]`
   Description: Here, subarray `1,2,3` has maximum 3, `2,3,1` has maximum 3, `3,1,4` has maximum 4, `1,4,5` has maximum 5, `4,5,2` has maximum 5, `5,2,3` has maximum 5, and `2,3,6` has maximum 6.
2. **Input:** array = `[8, 5, 10, 7, 9, 4, 15, 12, 90, 13]`, k = 4
   Output: `[10, 10, 10, 15, 15, 90, 90]`
   Description: Here, the maximum of each subarray of size 4 are 10, 10, 10, 15, 15, 90, 90 respectively.
3. **Input:** array = `[1,2,3,4,5]`, k = 3
   Output: `[3, 4, 5]`
   Description: Here, the maximum of each subarray of size 3 are 3, 4, 5 respectively.

Solution

The approach to solve this problem involves utilizing a deque (double-ended queue) to efficiently track the maximum elements in each window of size 'k' as we traverse the array. Initially, we populate the deque with indices of elements, ensuring that it always contains elements in decreasing order. This way, the front of the deque always holds the index of the current maximum element. As we move the window, we add new elements to the rear of the deque, removing those from the front that fall outside the current window. We also remove elements from the rear if they are smaller than the new element being added, as they are no longer potential maximums. This strategy ensures that for each window, we can quickly identify the maximum element, which is always at the front of the deque.

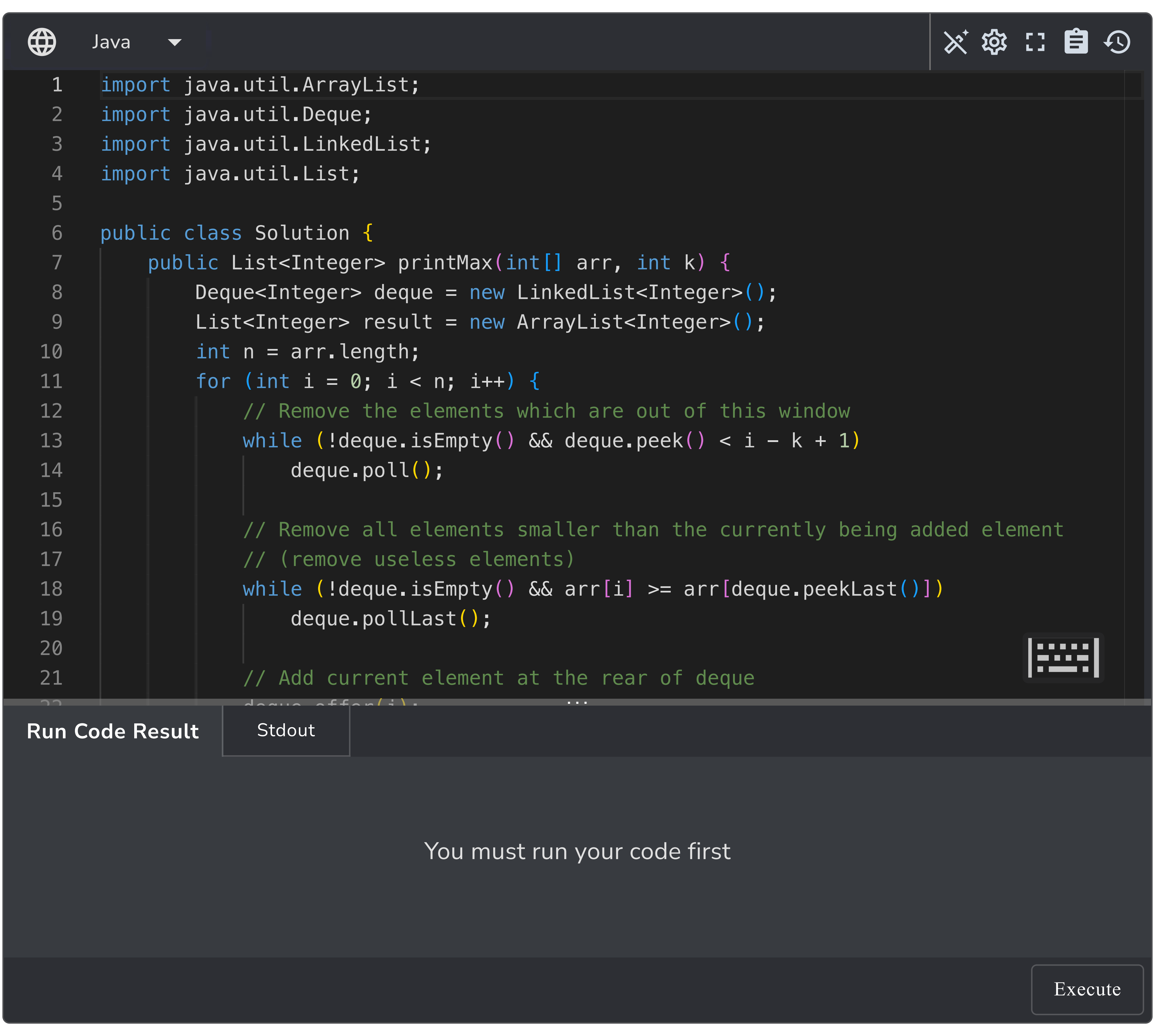
Here is the step-by-step algorithm.

1. **Initialize a Deque:** Create an empty deque (double-ended queue) that will be used to store the indices of array elements. This deque will help us maintain the potential maximum elements for each subarray.
2. **Process the First 'k' Elements:**
   - Iterate through the first 'k' elements of the array.
   - For each element, while the deque is not empty and the last element in the deque is less than or equal to the current element, remove the last element from the deque. This step ensures that the deque contains elements in decreasing order.
   - Add the current element's index to the rear of the deque.
3. **Process the Remaining Elements of the Array:**
   - For each element in the array starting from the 'k'th element:
     - Add the element at the front of the deque to the result, as it represents the maximum of the previous subarray.
     - Remove the indices from the front of the deque if they are out of the current window (i.e., if the index is less than the current index - 'k').
     - Similar to step 2, remove elements from the rear of the deque if they are smaller than or equal to the current element, as they cannot be the maximum for the current or future windows.
     - Add the current element's index to the rear of the deque.
4. **Return the Result:**
   - The result contains the maximum of each subarray of size 'k'.



Code

Here is how we can implement this algorithm:



```java
import java.util.ArrayList;
import java.util.Deque;
import java.util.LinkedList;
import java.util.List;

public class Solution {
    public List<Integer> printMax(int[] arr, int k) {
        Deque<Integer> deque = new LinkedList<Integer>();
        List<Integer> result = new ArrayList<Integer>();
        int n = arr.length;
        for (int i = 0; i < n; i++) {
            // Remove the elements which are out of this window
            while (!deque.isEmpty() && deque.peek() < i - k + 1)
                deque.poll();

            // Remove all elements smaller than the currently being added element
            // (remove useless elements)
            while (!deque.isEmpty() && arr[i] >= arr[deque.peekLast()])
                deque.pollLast();

            // Add current element at the rear of deque
```

You must run your code first

Time and Space Complexity

The above algorithm has a time complexity of $O(n)$, where n is the number of elements in the array. This is because each element in the array is pushed and popped from the queue at most once.

The space complexity of this algorithm is $O(k)$, where k is the size of the window. This is because the queue will hold at most k elements - these are the indices of the elements which are in the current window and may become the maximum element of the window as the window slides.

← Previous
Problem 6: Max Of All Subarrays Of Size 'K'

Next →
Introduction To LinkedList

☐ Mark as Completed